

System Specification Document

Machine Learning Operations

Basit, Lepori, Pierro, Tomasi, Ziurkalov

February 12th 2026

Contents

1	Problem Definition	2
1.1	Business Plan	2
1.2	Machine Learning Formulation	2
1.3	KPIs	2
2	Data Specification	3
2.1	Data Sources	3
2.2	Data Flow	3
2.3	Quality	3
2.4	Pre-processing	4
2.5	UML Component Diagram	4
3	Functional Requirements	4
3.1	Data Engineering Module	4
3.2	Process Mining Module	5
3.3	Machine Learning Module	5
3.4	User Interface (Dashboard)	5
4	Non-Functional Requirements	5
4.1	Privacy and Security	5
4.2	Performance and Efficiency	6
4.3	Reliability and Maintainability	6
4.4	Usability	6
5	Project Architecture	6
5.1	Training Phase	6
5.2	Validation Phase	6
5.3	Deployment Phase	7
5.4	Monitoring Phase	8
6	Risk Analysis	8
6.1	Risk Mitigation Strategies	9

Problem Definition

Excessive waiting time in hospitals is known to be a worldwide issue, influencing the medical staff ability to care for patients. Recognizing patterns between procedures and analyzing differences in time and type of day can help optimize waiting times and make hospitals significantly more efficient. Although every patient is provided with a unique ID and timestamp, the system lacks links between procedures and diagnoses.

This project aims to develop an intuitive and clear interface for reviewing the paths followed by patients, as well as a system to predict in advance and potentially shorten these paths. The system will be able to detect anomalies in waiting times and suggest modifications to treatment strategies by using a predictive model.

Business Plan

The primary objective of this project is to analyze and optimize the patient journey within a hospital Emergency Department (ED). By leveraging Process Mining techniques and a robust MLOps framework, we aim to uncover bottlenecks, visualize patient flows, and predict key performance indicators.

The project addresses the critical need for operational efficiency in healthcare, aiming to reduce waiting times and improve resource allocation while ensuring data privacy and system reliability through automated pipelines.

Machine Learning Formulation

To ensure clarity in optimization goals, the predictive engine is structured around a primary and a secondary target:

- **Primary Target (Regression):** Predict the *next-step waiting time* (in minutes). This directly addresses the bottleneck detection requirement and allows for granular optimization of the patient flow.
- **Secondary Target (Classification):** Predict the *next activity class*. This serves to reconstruct and anticipate the patient's clinical pathway variant.

This formulation moves away from the generic Length of Stay (LOS) prediction as the sole metric, focusing instead on actionable delays between specific procedures.

KPIs

The system's effectiveness is evaluated using the following operational Key Performance Indicators:

- **Data Pipeline Efficiency:** Zero critical errors during the ingestion of daily logs.
- **Model Accuracy (MAE):** Mean Absolute Error on the *next-step waiting time*, calculated on a temporal split (training on past months, evaluation on the most recent month).
- **Model Stability:** Drift detection based on performance degradation rather than just feature distribution.

Data Specification

Data Sources

The dataset currently used for development consists of a synthetic event log derived from an Emergency Department workflow at a hospital in Massachusetts (USA).

Data Access Strategy:

- *Development:* Data is provided locally via Docker volume mounting for rapid iteration.
- *Production:* The pipeline integrates with DVC (Data Version Control) to securely pull the specific, validated version of the dataset from a remote object storage at runtime. This eliminates dependency on local file systems and ensures traceability.

Data collection follows these structural steps:

1. Each patient is assigned a unique, anonymized identification code.
2. Timestamps are generated at the start and end of each activity to calculate the *waiting time*.

Data Flow

The data flow will describe the end-to-end pipeline through which hospital patient's activities will be recorded, ingested, processed, and made available for analysis.

1. **Data ingestion:** the CSV dataset containing all the required information will be extracted from the hospital system.
2. **Feature choice:** relevant columns of data will be determined to ensure efficiency.
3. **Pre-processing:** the dataset will undergo cleaning and pre-processing steps.
4. **Dataset Derivation:** a derived dataset that will be cleaned and will contain only relevant information will be generated.
5. **Visualization:** authorized clients will access the data through a Dashboard for real-time monitoring of clinical paths.

Quality

In order to ensure consistency between real-world clinical paths and the ones presented in the dataset, basic validation checks will be applied to identify potential issues. To ensure data reliability, the following aspects will be considered:

- **Completeness:** verify required columns exist and timestamps parse correctly.
- **Temporal consistency:** enforce per-encounter ordering rules (start : timestamp end : timestamp for each event).
- **Activity overlap:** identifying cases where the same patient might be simultaneously engaged in multiple activities.

- **Outlier detection:** checking for anomalous waiting times.
- **Encounter ID integrity:** ensuring that each encounter corresponds to a specific clinical path.

Pre-processing

Pre-processing will be fundamental to prepare data for training; the planned steps will include:

- **Data cleaning:** rows containing missing values will be deleted, along with incomplete timestamps.
- **Timestamp alignment:** timestamps will be ordered chronologically.
- **Dataset Derivation (Feature Selection):** only the attributes previously flagged as necessary will be retained, ensuring processing efficiency.

UML Component Diagram

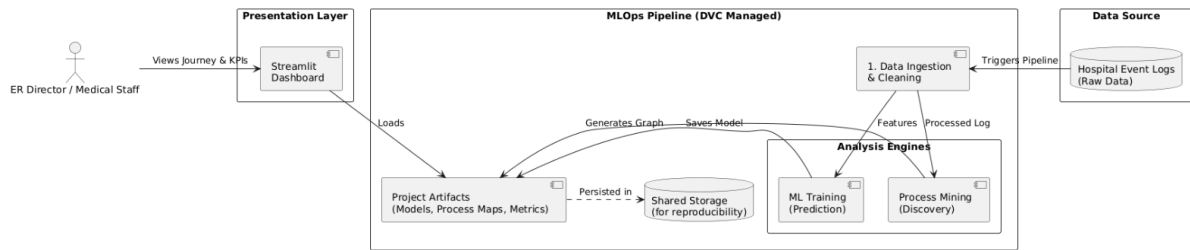


Figure 1: MLOps System Architecture and Data Flow

Functional Requirements

The system functionality is divided into four main modules: Data Engineering, Process Mining, Machine Learning, and User Interface.

Data Engineering Module

1. **Data Ingestion:** The system shall ingest event logs in standard formats (.csv or .xes) containing at minimum: *Encounter (case ID)*, *Activity Name*, and *Timestamp*.
2. **Data Cleaning:** The system must automatically filter out incomplete traces (e.g., missing timestamps) and correct formatting errors during the pre-processing stage.
3. **Data Versioning:** The system shall use DVC to track changes in the dataset, ensuring that every model training run is linked to a specific version of the data.

Process Mining Module

1. **Process Discovery:** The system shall generate visual process maps (Directly-Follows Graphs - DFG) representing the patient journey through the Emergency Department.
2. **Variant Analysis:** The system shall identify and display the most frequent paths (process variants) taken by patients.
3. **Bottleneck Detection:** The system must highlight transitions between activities that exceed a configurable time threshold, identifying potential delays in the hospital flow.

Machine Learning Module

1. **Training Pipeline:** The system shall provide an automated pipeline to train regression models (to predict waiting times) and classification models (to predict the next activity).
2. **Prediction Inference:** Given the current status of a patient, the system shall output the predicted remaining Length of Stay in minutes.
3. **Drift Detection:** The system shall monitor incoming data distributions. If significant concept drift is detected (e.g., MAE degradation), a visual warning (e.g., "Model Outdated").

User Interface (Dashboard)

1. **Visualization:** The system shall provide a web-based interactive dashboard (using Streamlit) accessible to medical staff.
2. **Filtering:** Users must be able to filter the process map and analytics by time range, days, or specific diagnosis groups.
3. **KPI Display:** The dashboard must display real-time Key Performance Indicators, such as Average Waiting Time and Total Patients in the system.
4. **Feedback Loop:** The interface shall provide a mechanism (e.g., "Report Inaccuracy" button) allowing medical staff to flag incorrect predictions. These flags will be logged to the experiment tracking system to prioritize data segments for future retraining cycles.

Non-Functional Requirements

These requirements define the quality attributes of the system, ensuring usability, reliability, and compliance.

Privacy and Security

- **Access Control:** Access to the dashboard and the underlying data must be restricted to authorized hospital personnel only.

Performance and Efficiency

- **Inference Latency:** The predictive model must return a result (prediction of waiting time) with low latency following a request to ensure real-time utility for staff.
- **Training Efficiency:** The model retraining pipeline should be optimized to run within a reasonable timeframe on standard hospital server infrastructure.

Reliability and Maintainability

- **Reproducibility:** The system architecture must ensure that any result can be reproduced by using the specific combination of code commit (Git) and data snapshot.
- **Containerization:** The application must be deployed via Docker containers to ensure consistency across development, testing, and production environments.
- **Modularity:** The codebase shall follow a modular structure (separation of `src/features`, `src/models`, `src/visualization`) to facilitate future updates and maintenance.

Usability

- **Interpretability:** The system must present results in a format understandable by non-technical medical staff.
- **Documentation:** Comprehensive documentation (README, Notebook) must be provided to allow hospital staff to manage the system.

Project Architecture

Training Phase

The training phase is designed to ensure reproducibility and robustness by leveraging the DVC pipeline defined in the system architecture.

Initially, the system ingests the EventLog.csv dataset. The data ingestion step triggers the `src/ingest_data.py` script, which handles cleaning and formatting.

For the model training, the dataset is split chronologically to respect the temporal nature of clinical pathways. Specifically, a 80/20 split is applied: the first 80% of encounters are used for training, and the remaining 20% are reserved for validation.

To ensure full reproducibility, every training run is linked to a specific commit hash and data snapshot.

Validation Phase

The validation phase aims to quantitatively assess the performance of the predictive models before they are served to the Streamlit dashboard.

Model evaluation is performed using standard regression metrics for the LoS prediction (Mean Absolute Error) and classification metrics for next-event prediction (e.g., Accuracy, F1-Score).

A model is considered valid for deployment only if:

1. It successfully produces the serialized artifacts (pickled models).
2. The inference latency remains within the acceptable threshold for real-time interaction.
3. The generated Process Map is successfully rendered and stored as a PNG image.

Deployment Phase

The deployment strategy relies on containerization to ensure consistency across development and production environments. This project is deployed as a web service. The following table summarizes the deployment actions.

Component / Task	Deployment Actions and Configuration
Containerization	The application is packaged using a <code>Dockerfile</code> based on the <code>python:3.9-slim</code> image. System dependencies such as <code>graphviz</code> are installed to support Process Mining visualizations.
Dependency Management	Python dependencies are frozen in <code>requirements.txt</code> and installed via <code>pip</code> during the build process to guarantee environment stability.
Service Orchestration	The container exposes port 8501. The entry point is configured to launch the Streamlit dashboard via <code>streamlit run src/dashboard.py</code> .
Model Artifacts	Trained models (pickled artifacts) and Process Maps (generated PNGs) are loaded into the container at runtime. This ensures the application serves the latest validated version tracked by DVC.
Post-Deployment Verification	A post-deployment verification checks if the <code>patient_journey_dfg.png</code> loads correctly and if the prediction engine responds to a sample input vector without errors.

Table 1: Deployment Phase Component Configuration and Tasks

Monitoring Phase

The monitoring phase constitutes the primary interface between the analytical backend and the medical staff, enabling real-time interaction with the system’s predictive capabilities. This phase is implemented through a web-based dashboard built on the Streamlit framework, designed to ensure data accessibility and provide actionable insights into the patient journey. The system’s monitoring infrastructure is organized into three core functional areas:

Process Visualization

To support retrospective analysis, the dashboard renders the "Hospital Process Flow" using the DFG generated during the discovery stage. This visualization allows hospital administrators to visually inspect the sequence of activities, identifying potential bottlenecks, loop structures, or inefficient transitions within the Emergency Department workflow. The graph is dynamically loaded, ensuring that the visual representation always reflects the most recent data snapshot processed by the DVC pipeline.

Real-time Prediction Interface

The dashboard features an interactive sidebar that functions as a simulation environment for medical staff. Users can configure a specific patient profile by selecting a sequence of performed procedures (e.g., Depression screening, X-Ray, Blood Test) from a predefined list. Upon triggering the prediction engine, the backend model processes the input vector to estimate the remaining Length of Stay. The result is displayed as a clear, high-visibility metric indicating the "Estimated Discharge Time" in minutes, supported by contextual information regarding the model version used for inference.

Operational Integrity and Error Handling

To ensure robustness in a production environment, the system includes sophisticated feedback mechanisms. The application validates the availability of essential artifacts—specifically the serialized machine learning models and the generated process maps—before attempting to render them. In cases where artifacts are missing (e.g., if the training pipeline has not been executed), the interface provides specific warning messages (e.g., "Process Map not found"), preventing system crashes and guiding the user towards the necessary troubleshooting steps.

Risk Analysis

The implementation of a predictive monitoring system in a critical environment like an Emergency Department entails specific risks regarding data integrity, model reliability, and operational continuity. The following table summarizes the identified risks and their potential impact.

Risk	Description
Data Quality & Integrity	Incomplete or noisy event logs (e.g., missing timestamps, incorrect activity labels) may lead to distorted Process Maps and unreliable predictions. Dependency on manual data entry by hospital staff is a critical vulnerability.
Concept Drift	Patient flow patterns can change rapidly due to external factors (e.g., seasonal flu epidemics, public health emergencies). The trained model may become obsolete if the distribution of live data diverges significantly from the training set.
Pipeline Failure	The dependency on the DVC pipeline and Docker infrastructure means that failures in data ingestion scripts or container orchestration could render the dashboard unavailable during critical shifts.
Scalability & Concurrency	Streamlit is designed for prototyping. In a real scenario with concurrent access by multiple departments, the application may suffer from latency or connection drops.

Table 2: Project Risk Analysis Matrix

Risk Mitigation Strategies

To address these risks, the system incorporates:

- **Automated Validation:** The ingestion module implements checks for timestamp consistency and missing values before processing.
- **Drift Monitoring:** A dedicated module monitors performance metrics on new data to trigger alerts when retraining is required.
- **Fallback Mechanisms:** In case of model failure, the dashboard degrades gracefully, showing only historical descriptive statistics (Process Mining) without attempting unreliable predictions.